PROYECTO INTEGRADOR · HITO II

ATE: Analítica de Tareas Educativas

Bitácora de avance y evaluación multidimensional

Módulo 4: Big Data y MLOps · Bootcamp Python para Ciencia de Datos

Cierre del Sprint 1 · Bitácora de desarrollo, Design Thinking y evaluación financiera

Integrante	Código	Rol
Julio César Flores Fuentes	04973972-7	Project Manager
Mauricio Enrique Castro Flores	06018072-1	Product Owner
Claudia Patricia Carballo Larios	01747225-0	Scrum Master

San Salvador, 3 de noviembre de 2026

Índice

1. Avance del Sprint 1 3
2. Bitácora de desarrollo 5
2.1 Cronología 5
2.2 Hallazgos en los datos 5
2.3 Cuellos de botella y cómo los resolvimos 6
2.4 Decisiones de arquitectura 7
3. Resultados técnicos 8
3.1 Dask frente a Pandas 8
3.2 Historial del estudiante con PySpark 9
3.3 MongoDB y S3 9
3.4 Modelos de aprendizaje automático y profundo 10
3.5 Calidad del código 13
4. Evaluación de usuario con Design Thinking 14
4.1 Empatizar 14
4.2 Definir 14
4.3 Idear 15
4.4 Prototipar 15
4.5 Evaluar 16
5. Evaluación tecnológica, económica y financiera 18
5.1 Viabilidad técnica 18
5.2 Supuestos del modelo de negocio 18
5.3 Flujo de caja y ROI a 12 y 24 meses 19
6. Riesgos y próximos pasos 21
6.1 Riesgos actualizados 21
6.2 Plan del Sprint 2 21
7. Declaración de uso de inteligencia artificial 22
Referencias 23

Este documento es la bitácora de avance del Proyecto Integrador ATE al cierre del Sprint 1. Registra lo construido, lo que descubrimos en los datos, los obstáculos y cómo los resolvimos, y evalúa el proyecto en los tres ejes que pide la Guía de Planificación (Bootcamp de Python para Ciencia de Datos, 2026a): bitácora de desarrollo, evaluación de usuario con Design Thinking y evaluación tecnológica, económica y financiera.

Resumen del avance. El pipeline de datos funciona de punta a punta sobre 10 millones de entregas sintéticas: Dask limpia y agrega, PySpark calcula el historial de cada estudiante, MongoDB guarda las tareas con validación e índices y S3 guarda archivos y artefactos. Los tres mejores modelos (regresión logística, Random Forest y una red MLP) casi duplican el F1 del baseline en la clase «tardía» (0,48 frente a 0,25), con un AUC ROC de 0,71. El repositorio tiene 26 pruebas pytest que pasan sin tocar la red. En el escenario base, ATE recupera la inversión en el mes 14 y alcanza un ROI de 4,6 veces a los 24 meses.

1. Avance del Sprint 1

El Sprint 1 (26 al 31 de octubre) tenía como meta dejar los datos, el almacenamiento y los modelos funcionando. Las historias se trabajaron en ramas `feature/HU-xx` integradas a `main` con *merge* explícito, para que el historial muestre cada entrega.

Tabla 1

Estado de las historias del Sprint 1

ID	Historia	Estado	Evidencia en el repositorio
HU-05	Dataset sintético reproducible	Terminada	`src/ingesta/generador.py`, `src/utils/calendario.py`, `tests/test_datos.py`, `tests/test_calendario.py`
HU-06	Limpieza y agregación con Dask	Terminada	`src/ingesta/dask_pipeline.py`, `reports/tiempos_dask_vs_pandas.csv`
HU-07	Historial del estudiante con PySpark	Terminada	`src/ingesta/spark_etl.py`, `docs/spark_vs_dask.md`, `tests/test_spark.py`
HU-08	MongoDB con validación	Terminada en código y pruebas; carga a Atlas pendiente	`src/almacenamiento/mongo_repo.py`, `tests/test_mongo_repo.py`
HU-09	Almacén S3	Terminada	`src/almacenamiento/almacen_s3.py`, `tests/test_almacen_s3.py`
HU-10	Modelo de riesgo de entrega tardía	Terminada	`src/modelos/`, `reports/metricas_modelos.csv`
HU-11	Clases POO y pruebas (inicio)	Terminada la parte del Sprint 1	`src/ingesta/base.py`, `src/modelos/base.py`, `tests/`

ID	Historia	Estado	Evidencia en el repositorio
HU-16	Bitácora y evaluación	Terminada (este documento)	`src/utils/finanzas.py`, `docs/decisiones.md`

Nota. La Definición de Done de la propuesta pide que cada historia entre por un pull request revisado por otro integrante. Mientras el repositorio vive solo en local, las ramas se integran con git merge --no-ff; al publicarlo en GitHub, el Sprint 2 se trabajará con pull requests y revisión cruzada.

2. Bitácora de desarrollo

2.1 Cronología

Tabla 2

Bitácora del Sprint 1

Fecha	Qué pasó	Qué decidimos
25 oct	Se aprobó la propuesta y se creó el repositorio con la estructura Cookiecutter Data Science	Ramas por historia de usuario; `.env` fuera de git desde el primer commit
28 oct	El generador produjo 10 millones de entregas (1,5 GB en CSV, 124 MB en Parquet)	Guardar los datos fuera de OneDrive y fuera del repositorio; se regeneran con la semilla
28 oct	Pandas no pudo cargar 2,5 millones de filas con la RAM libre de la laptop	Dask para la limpieza y las agregaciones (decisión 005)
28 oct	Spark 4 no lee archivos locales en Windows sin `winutils.exe`	Leer cada Parquet con Pandas y pasarlo a Spark por lotes (decisión 006)
28 oct	MongoDB Atlas rechazó la conexión desde la laptop nueva (`TLSV1_ALERT_INTERNAL_ERROR`)	Las pruebas usan mongomock; la carga a Atlas espera a que se agregue la IP en *Network Access*
28 oct	El sistema detuvo el ETL de Spark por falta de memoria a mitad del proceso	Repetirlo con 1 GB para el *driver* y con los navegadores cerrados
2 nov	El ETL de Spark avanzaba tan lento que habría tardado más de una hora, y después se quedó sin memoria del *driver*	Pasar el texto a Spark como `object` y liberar cada lote al terminarlo: 10 M de filas en 6 min 40 s
2 nov	Se entrenaron el baseline, tres modelos clásicos y el MLP con 358 448 tareas de 1 551 estudiantes	La API usará la regresión logística: rinde igual que el MLP, es interpretable y no necesita TensorFlow

2.2 Hallazgos en los datos

Los datos son sintéticos (decisión 004): el generador fija la semilla y define el riesgo de entrega tardía con reglas explícitas. Por eso, los hallazgos sirven para comprobar que el pipeline recupera esas reglas, no para sacar conclusiones sobre estudiantes reales.

- **Volumen:** 10 millones de entregas de 40 000 estudiantes (250 tareas cada uno en el año escolar 2026), en 60 centros y 10 distritos. Los archivos crudos suman 10 006 680 filas porque incluyen los duplicados inyectados.

- **Defectos inyectados:** el 0,2 % de las filas trae duplicados, notas fuera de rango o texto mal escrito («Matematica», «REGULAR»). La limpieza con Dask los corrige o descarta, y una prueba lo verifica.
- **Clase desbalanceada:** el 26 % de las entregas cerradas son tardías. Por eso medimos F1, precisión y exhaustividad de la clase «tardía» y no solo la exactitud.
- **Modalidad:** la tasa de entrega tardía es de 24,6 % en la modalidad regular, 29,9 % en fin de semana, 32,2 % en nocturna y 34,3 % a distancia.
- **Asuetos:** cuando el plazo cruza un asueto o vacaciones, la tasa sube de 25,5 % a 30,6 %. Es la razón de calcular los plazos con el calendario salvadoreño.
- **Materias:** Matemática en bachillerato es la combinación con más retrasos (33,2 % en primer año) y Educación Física la de menos (cerca de 20 %).
- **Pendientes:** al 28 de octubre quedan unas 12 000 tareas pendientes por centro, que son las que el modelo debe puntuar.

2.3 Cuellos de botella y cómo los resolvimos

Tabla 3

Obstáculos técnicos del Sprint 1

Obstáculo	Causa	Solución	Resultado
Pandas se queda sin memoria	La laptop tiene 8 GB y, con VS Code y dos navegadores abiertos, solo 0,6 a 1,4 GB libres	Dask con `dtype` declarados, columnas categóricas y `blocksize="64MB"`	10 M de filas limpias y agregadas en 45 s
Spark no lee ni escribe en Windows	Spark 4 usa el sistema de archivos de Hadoop, que en Windows necesita `winutils.exe` y `hadoop.dll`	En Windows cada archivo se lee con Pandas y pasa a Spark; en Linux o Docker se usa `spark.read.parquet`	Mismo resultado, sin instalar binarios de terceros
ETL de Spark muy lento	PySpark 4.2 aún no convierte el texto de Pandas 3, que se guarda en arreglos Arrow fragmentados (`Cannot convert pyarrow.lib.ChunkedArray`); Spark caía a la conversión fila por fila	Convertir las columnas de texto a `object` antes de `createDataFrame`	Un lote de 500 000 filas pasa a Spark en 10 s; el ETL completo dejó de tardar más de una hora
Proceso de Spark detenido por memoria	Las ventanas por estudiante y la conversión a Pandas compiten por la poca RAM libre	*Driver* de 1 GB, procesamiento por lotes y muestra de modelado del 4 % de estudiantes	10 M de filas en 6 min 40 s con 1,5 GB para el *driver*
Atlas rechaza la conexión	La lista de acceso de IP de Atlas solo tenía la IP de otra computadora	Agregar la IP actual en *Network Access*; mientras tanto, pruebas con mongomock	Pendiente de confirmar la IP
Riesgo de fuga de datos	Un estudiante tiene unas 250 tareas; si se reparten entre entrenamiento y prueba, el modelo lo «reconoce»	División por estudiante, historial con desfase de 21 días y tasa por docente calculada sin la muestra de modelado	Métricas más bajas pero honestas

2.4 Decisiones de arquitectura

Cada decisión está registrada en `docs/decisiones.md` con su contexto, alternativas y consecuencias, como pide la guía (Bootcamp de Python para Ciencia de Datos, 2026b, apéndice B).

Tabla 4

Decisiones registradas hasta el Hito II

N.°	Decisión	Por qué
001	Un solo backend en FastAPI	Atiende las tareas y el modelo; todo el código evaluado queda en Python
002	MongoDB como único almacén de la aplicación	Las tareas y su seguimiento son documentos semiestructurados y Atlas ya estaba configurado
003	Frontend React como PWA	Un solo código para Android, iOS, Huawei y escritorio, sin tiendas de aplicaciones
004	Datos sintéticos con reglas documentadas	La guía prohíbe datos personales reales y ninguna escuela puede ceder su historial
005	Dask para limpiar y agregar	Pandas no cabe en la RAM libre a partir de 2,5 M de filas
006	PySpark para el historial del estudiante	Las funciones de ventana por grupo y el *join* con *broadcast* son el punto fuerte de Spark
007	Validación por estudiante y F1 de la clase «tardía»	Evita métricas infladas y no se deja engañar por el desbalance
008	Validación Pydantic e índices compuestos en MongoDB	Ningún documento sucio llega a la base y la consulta principal usa índice
009	S3 con prefijo propio y moto en las pruebas	Se comparte el bucket de la Práctica NoSQL sin mezclar archivos; las pruebas no usan la red
010	MLP en Keras como modelo de aprendizaje profundo	Los datos son tabulares; una CNN o una LSTM no tendrían justificación

3. Resultados técnicos

3.1 Dask frente a Pandas

Medimos la misma limpieza y las mismas cuatro agregaciones con Pandas y con Dask, de 1 a 20 archivos de unas 500 000 filas cada uno, y registramos el tiempo y el pico de memoria adicional del proceso.

Tabla 5

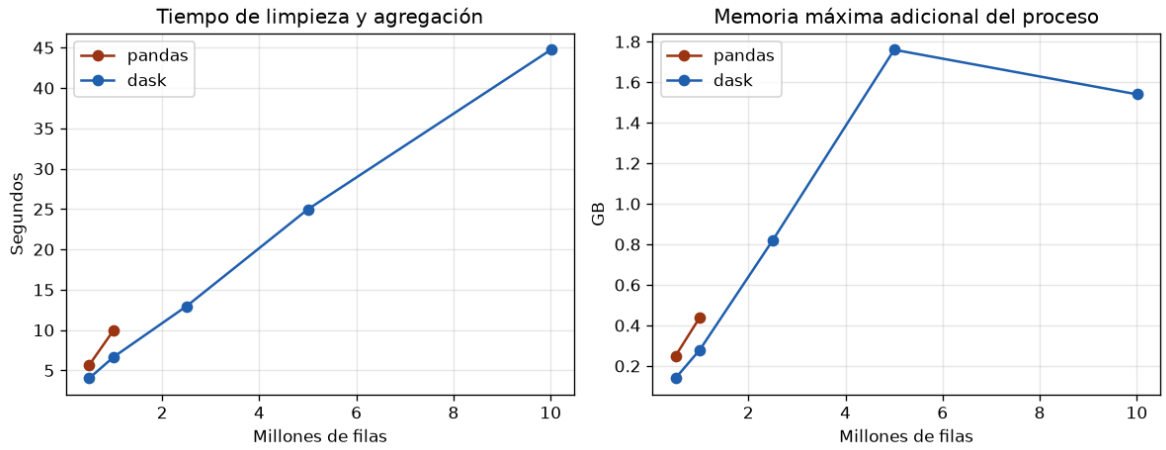
Tiempo y memoria de Dask frente a Pandas

Filas	Pandas (s)	Dask (s)	Memoria extra Pandas (GB)	Memoria extra Dask (GB)
0,5 M	5,7	4,1	0,25	0,14
1,0 M	10,0	6,7	0,44	0,28
2,5 M	No cabe	13,0	—	0,82
5,0 M	No cabe	25,0	—	1,76
10,0 M	No cabe	44,7	—	1,54

Nota. «No cabe» significa que el script no lanzó Pandas porque la memoria estimada superaba la RAM libre en ese momento (0,6 a 1,4 GB). Fuente: reports/tiempos_dask_vs_pandas.csv.

Figura 1

Tiempo de procesamiento de Dask y Pandas según el número de filas



Nota. Elaboración propia con python -m src.ingesta.dask_pipeline --benchmark.

Dask divide el trabajo en particiones y lo ejecuta en paralelo con un grafo de tareas (Rocklin, 2015). Con 1 millón de filas Dask fue 1,5 veces más rápido y usó la mitad de memoria. El tiempo de Dask crece casi en línea recta con el volumen, porque procesa una partición por núcleo y nunca carga todo a la vez. Elegimos `blocksize="64MB"`, que deja más o menos una partición por archivo: con particiones más pequeñas, el costo de coordinar tareas supera al cálculo.

3.2 Historial del estudiante con PySpark

Spark ejecuta un plan optimizado por su motor Catalyst, que es el mismo en una laptop o en un clúster (Zaharia et al., 2016). Corre en modo `local[*]` y calcula, para cada tarea, cómo le fue al estudiante en las tareas anteriores cuyo resultado ya se conocía. Usa una ventana por estudiante ordenada por fecha con `rangeBetween(unboundedPreceding, -21)` y une la tasa de entregas tardías de cada docente con un `join` de tipo *broadcast*. Todo es perezoso hasta las acciones `count()` y `toPandas()`; el plan físico se guarda en `reports/plan_spark.txt`.

Tabla 6

Ejecución del ETL de PySpark sobre el volumen completo

Medida	Valor
Filas procesadas	10 000 000, en 20 lotes de unas 500 000 filas
Paso 1: tasa de entregas tardías de 480 docentes	244 s
Paso 2: ventanas, *join* y muestra de modelado	155 s
Tiempo total	6 min 40 s, con 1,5 GB para el *driver*
Muestra de modelado (4 % de los estudiantes)	387 750 tareas de 1 551 estudiantes: 358 448 con resultado conocido y 29 302 pendientes

Nota. Ejecución en una laptop con Windows 11 y 8 GB de RAM, el 2 de noviembre de 2026. La muestra se elige con un hash del número de estudiante, así que siempre son los mismos estudiantes.

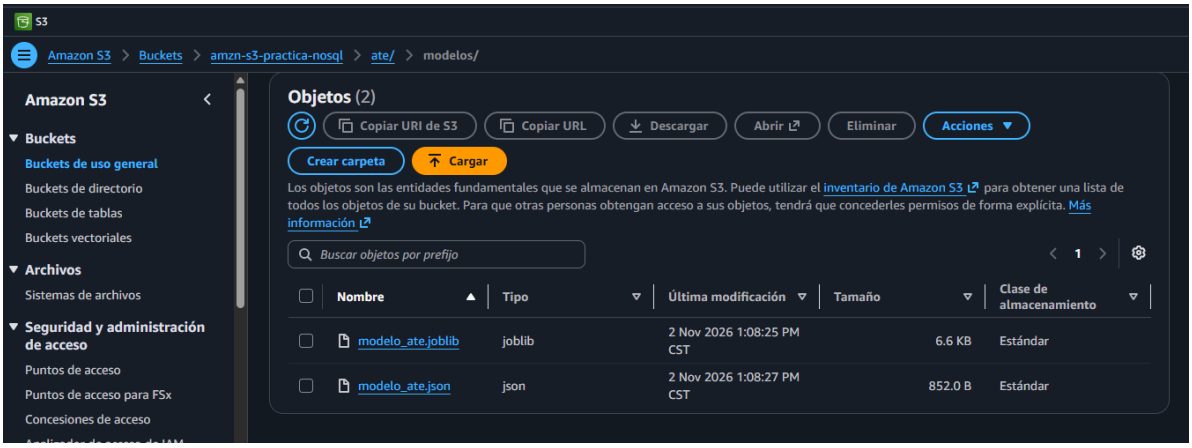
La comparación razonada con Dask está en `docs/spark_vs_dask.md`. En resumen: Dask conviene para trabajo por partición y agregaciones con código de Pandas; Spark, para ventanas, uniones grandes y para crecer a un clúster real sin reescribir el código.

3.3 MongoDB y S3

- **MongoDB:** cada documento pasa por el modelo Pydantic `TareaDocumento` (con `extra="forbid"`) antes de `insert_one` o `insert_many`. Hay tres índices: `id_entrega` único, `(numero_estudiante, fecha_limite)` y `(estado, fecha_limite)`. Las consultas usan `$gt` y `$lt`, y el historial de cambios de cada tarea vive dentro del documento. Las pruebas usan mongomock. La carga de una muestra de 50 000 tareas en Atlas y la medición con `explain` esperan a que se habilite la IP de la laptop.
- **S3:** `AlmacenS3` sube y descarga archivos con `upload_file` y `download_file`, lee las credenciales solo de variables de entorno y traduce los errores 403 y 404 a excepciones propias. Usa el bucket de la Práctica NoSQL bajo el prefijo `ate/`. Las pruebas usan moto y nunca tocan la red; con `S3_ENDPOINT_URL` el mismo código funciona con MinIO. El 2 de noviembre subimos al bucket el modelo, sus metadatos, el dataset de modelado y la tabla de métricas, y comprobamos que el archivo descargado es idéntico al original.

Figura 2

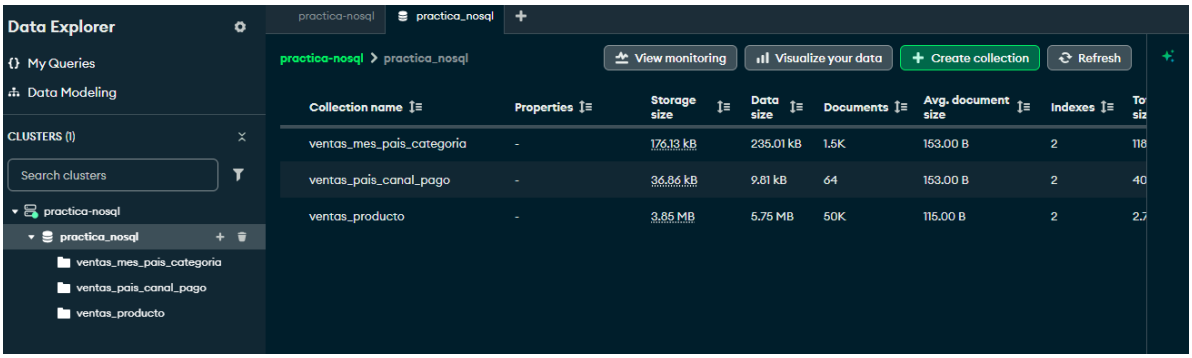
Artefactos de ATE en el bucket de S3, carpeta ate/modelos/



Nota. Captura de la consola de AWS del 2 de noviembre de 2026. En ate/processed/ está dataset_modelo.parquet (5,2 MB) y en ate/reportes/, metricas_modelos.csv; las tres capturas están en docs/capturas/.

Figura 3

Clúster practica-nosql de MongoDB Atlas que usará ATE



Nota. Captura del Data Explorer de Atlas. Las colecciones que se ven son de la Práctica NoSQL; ATE usará el mismo clúster con su propia base de datos, ate, en cuanto se habilite la IP del equipo.

3.4 Modelos de aprendizaje automático y profundo

El objetivo es estimar la probabilidad de que una tarea se entregue tarde. Todos los modelos comparten el contrato `ModeloBase` (patrón Strategy) y el mismo preprocesador dentro de un `Pipeline`: imputación y escalado de las variables numéricas y codificación *one-hot* de las categóricas. Así, el imputador y el escalador se ajustan solo con los datos de entrenamiento de cada pliegue. Los modelos clásicos usan scikit-learn (Pedregosa et al., 2011). La búsqueda de hiperparámetros usa `GridSearchCV` con `StratifiedGroupKFold` agrupado por estudiante.

Entrenamos con la muestra que produce Spark: 358 448 tareas con resultado conocido, de 1 551 estudiantes. La dividimos por estudiante en 80 % para entrenamiento (286 683 tareas) y 20 % para prueba (71 765). El 26,2 % de las entregas son tardías. Los modelos pertenecen a familias distintas, como pide la guía:

- **Baseline:** `DummyClassifier` estratificado, que predice al azar respetando la proporción de clases. Es la vara mínima.
- **Regresión logística** con regularización L2 y pesos de clase balanceados: modelo lineal, de poca varianza e interpretable. Se buscó el parámetro `C`.
- **SVM lineal** (`LinearSVC`) calibrada con una sigmoide para obtener probabilidades: busca el hiperplano de máximo margen. Se buscó `C`.
- **Random Forest** de 150 árboles: un *ensemble* no lineal que reduce la varianza al promediar árboles. La profundidad máxima y el mínimo de muestras por hoja actúan como regularización contra el sobreajuste.
- **MLP en Keras:** dos capas ocultas (64 y 32 neuronas), *dropout* de 0,3, *early stopping* sobre la pérdida de validación y pesos de clase.

Tabla 7

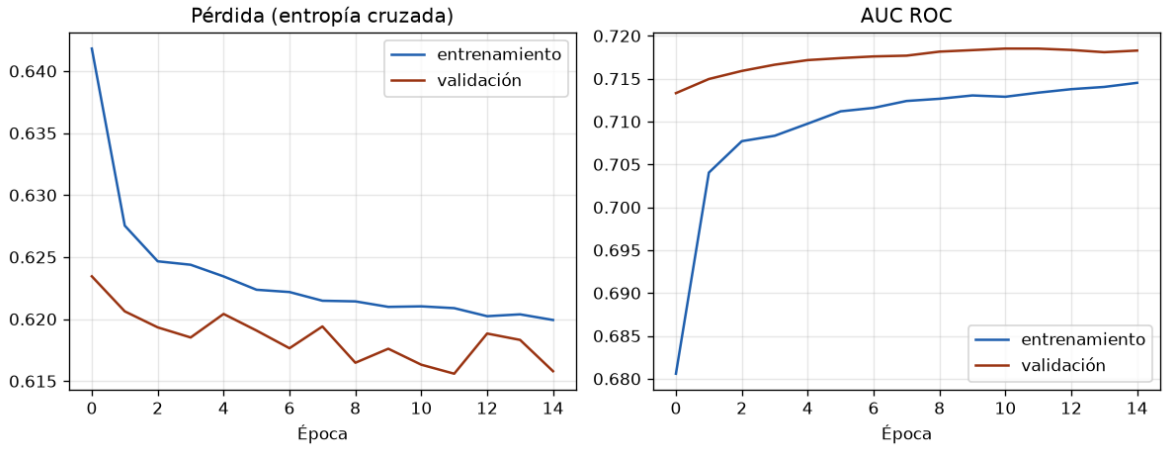
Comparación de modelos en el conjunto de prueba (clase positiva: entrega tardía)

Modelo	F1	Precisión	Exhaustividad	AUC ROC	AUC PR	Exactitud	Hiperparámetros elegidos
MLP (Keras)	0,483	0,395	0,621	0,709	0,463	0,663	64-32, *dropout* 0,3; paró en la época 15
Regresión logística	0,481	0,403	0,596	0,707	0,458	0,674	C = 1,0
Random Forest	0,480	0,406	0,586	0,709	0,462	0,678	profundidad 14; 80 muestras por hoja
SVM lineal calibrada	0,283	0,593	0,186	0,707	0,458	0,762	C = 0,01
Baseline	0,255	0,251	0,259	0,498	0,253	0,617	—

Nota. Umbral de 0,5 en todos los modelos. El F1 de la validación cruzada agrupada fue de 0,498 para la regresión logística, 0,497 para Random Forest y 0,293 para la SVM. Fuente: reports/metricas_modelos.csv.

Figura 4

Curvas de aprendizaje del MLP



Nota. La pérdida y el AUC de validación quedan mejor que los de entrenamiento porque el dropout solo actúa al entrenar.

Interpretación. Los tres mejores modelos empatan: F1 entre 0,480 y 0,483 y AUC ROC entre 0,707 y 0,709, una diferencia menor que la variación esperable entre particiones. Desde el compromiso entre sesgo y varianza, la explicación es que la señal es casi aditiva: el generador suma efectos de modalidad, plazo, asueto, materia y propensión del estudiante, y un modelo lineal ya los captura. Random Forest y el MLP tienen más capacidad, pero no encuentran interacciones que aprovechar. El techo de 0,71 de AUC viene del azar que el generador incluye a propósito: ningún modelo puede predecir ese ruido.

La SVM enseña algo sobre el umbral. Tiene el mismo AUC que la regresión logística, pero su F1 cae a 0,28. La calibración devuelve probabilidades cercanas a la frecuencia real (26 %), así que con un umbral de 0,5 casi no lanza alertas: solo detecta el 19 % de las entregas tardías. Con el umbral ajustado rendiría como la logística. El umbral es una decisión del negocio, no un detalle técnico.

El MLP es un resultado negativo bien argumentado. No supera al modelo lineal, converge en unas 10 épocas y el *early stopping* lo detiene en la 15 (Figura 4). Con datos tabulares y una señal aditiva, la red no aporta.

Decisión. La API usará la regresión logística: rinde igual, entrena en 21 segundos, pesa 7 KB, no necesita TensorFlow en la imagen Docker y sus coeficientes permiten explicar a la coordinación por qué una tarea tiene riesgo.

Tabla 8

Análisis de errores de la regresión logística por subgrupo

Grupo	Tareas	Tasa real de entrega tardía	F1	Exhaustividad	AUC ROC
Modalidad a distancia	2 807	33,1 %	0,543	0,753	0,697
Modalidad regular	56 451	24,8 %	0,469	0,565	0,705

Grupo	Tareas	Tasa real de entrega tardía	F1	Exhaustividad	AUC ROC
Con historial previo	65 480	25,4 %	0,493	0,606	0,717
Sin historial previo	6 285	24,2 %	0,354	0,482	0,563
Proyectos	10 455	28,4 %	0,508	0,692	0,697
Guías	14 478	21,5 %	0,440	0,477	0,719

Nota. Fuente: reports/analisis_errores.csv.

El error más importante es el **arranque en frío**: en las primeras tres semanas el estudiante no tiene historial y el AUC baja de 0,72 a 0,56, casi al azar. En esas semanas, ATE mostrará la tasa de la sección y del docente en vez de una alerta individual. La precisión de 0,40 significa que 4 de cada 10 alertas corresponden a un retraso real. Es aceptable para un aviso barato, como un recordatorio, y es otra razón para que la alerta nunca se use para calificar ni sancionar.

3.5 Calidad del código

- **POO:** dos clases base abstractas, `EtapaPipeline` (para `LimpiezaDask` y `HistorialSpark`) y `ModeloBase` (para los modelos), con herencia, encapsulamiento y `__repr__`. El patrón Strategy permite cambiar de modelo sin tocar el código que entrena o evalúa.
- **Pruebas:** 26 pruebas pytest independientes. Incluyen casos de error con `pytest.raises`, calidad de datos (determinismo del generador, reglas de fechas, limpieza de defectos), mocks de MongoDB (mongomock) y S3 (moto), la ventana de Spark y el modelo financiero.
- **Docstrings:** todas las funciones y clases públicas tienen docstrings en estilo Google.
- **Git con Gitflow:** `main` guarda solo versiones estables y etiquetadas; `develop` es la rama donde convergen todos los cambios; cada historia se trabaja en una rama `feature/HU-xx-nombre` que sale de `develop` y vuelve a ella. Ya están creadas `feature/HU-12-api-docker` para Mauricio Castro y `feature/HU-14-readme-calidad` para Claudia Carballo. Los commits son atómicos y con mensajes descriptivos, y el `.gitignore` excluye `.env`, `__pycache__/` y los archivos `.csv` (salvo la muestra mínima y las tablas de resultados).

Salida de `pytest -v` del 2 de noviembre de 2026:

```
$ pytest
tests/test_almacen_s3.py    ....    [4 pruebas]
tests/test_calendario.py    .....   [5 pruebas]
tests/test_datos.py         ....    [4 pruebas]
tests/test_finanzas.py      ..      [2 pruebas]
tests/test_modelos.py       .....   [5 pruebas]
tests/test_mongo_repo.py    .....   [5 pruebas]
tests/test_spark.py         .       [1 pruebas]
==================== 26 passed in 55.36s ====================
```

4. Evaluación de usuario con Design Thinking

Aplicamos las cinco etapas del marco del Hasso Plattner Institute of Design at Stanford (2010): empatizar, definir, idear, prototipar y evaluar. En este hito, la empatía se apoya en lo que el Product Owner recogió de familias y docentes y en el contexto de la propuesta; la evaluación con usuarios reales está planificada para el Sprint 2.

4.1 Empatizar

Tabla 9

Mapa de empatía de los usuarios de ATE

Usuario	Qué dice	Qué hace hoy	Qué siente	Dolor principal
Madre, padre o representante	«Me entero de que no entregó cuando ya le pusieron la nota»	Revisa cuadernos y grupos de WhatsApp de la sección	Frustración e impotencia	No tiene una vista única de lo pendiente
Estudiante	«No sabía que era para hoy»	Anota tareas en el cuaderno o confía en la memoria	Ansiedad al acercarse la fecha	Las tareas están dispersas entre materias y canales
Docente	«Paso lista de tareas a mano y nadie la revisa»	Registra entregas en hojas de cálculo o en papel	Sobrecarga administrativa	Repetir la misma información por varios canales
Coordinación académica	«Nos damos cuenta del problema a fin de trimestre»	Revisa notas cuando el período ya cerró	Reacciona tarde	No ve dónde se acumulan los retrasos hasta que es tarde

Nota. Las frases son representativas de las necesidades que el Product Owner recogió para la propuesta; no son citas textuales de personas identificables.

4.2 Definir

Formulamos el punto de vista de cada usuario y las preguntas «¿cómo podríamos…?» que guían el diseño:

- **Familias:** una madre que trabaja todo el día necesita ver en un minuto qué tareas faltan, porque hoy se entera tarde y no puede ayudar a tiempo. ¿Cómo podríamos mostrarle lo pendiente sin pedirle crear una cuenta ni dar datos personales?
- **Estudiantes:** un estudiante de bachillerato necesita una sola lista de tareas ordenada por fecha, porque las tiene repartidas entre materias. ¿Cómo podríamos evitar que una tarea con plazo corto o que cruza un asueto lo tome por sorpresa?
- **Coordinación:** la coordinación necesita saber qué tareas tienen riesgo alto de retraso antes de la fecha límite. ¿Cómo podríamos avisarle a tiempo sin que el aviso se use como sanción?

4.3 Idear

Tabla 10

Ideas evaluadas y decisión

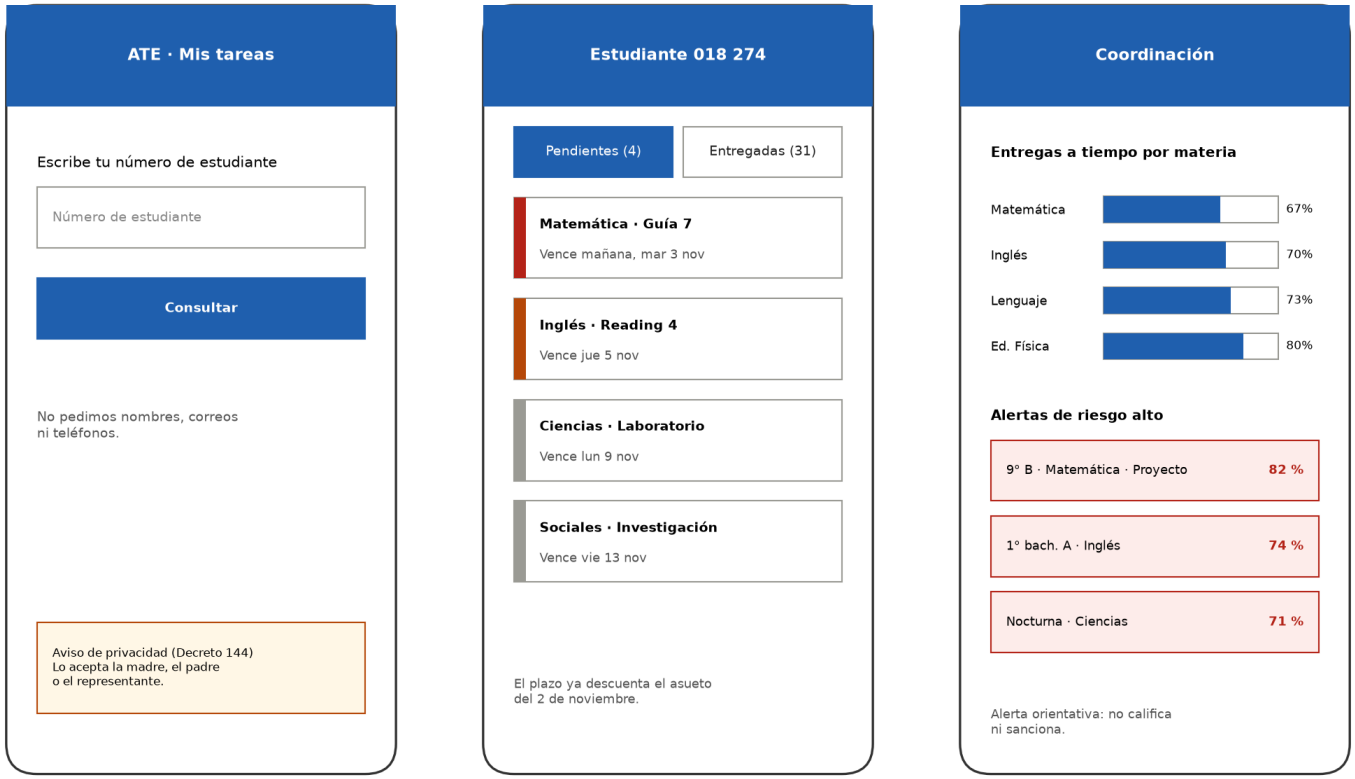
Idea	Decisión	Motivo
Cuentas con usuario y contraseña para las familias	Descartada	Añade fricción y obliga a guardar datos personales; se usa solo el número de estudiante (Asamblea Legislativa de El Salvador, 2024)
Aplicaciones nativas para Android e iOS	Descartada	Tres códigos y tiendas de aplicaciones; Huawei no tiene Google Play. Se elige una PWA
Lista de pendientes y entregadas por número de estudiante	Elegida	Responde al dolor principal de familias y estudiantes
Indicador de riesgo de entrega tardía	Elegida	Permite intervenir antes de la fecha límite
Plazos calculados con asuetos y modalidad	Elegida	Evita fechas límite en días sin clases, una causa medida de retraso
Recordatorios por WhatsApp o SMS	Pospuesta	Costo por mensaje y consentimiento adicional; queda como trabajo futuro
Ranking de estudiantes por riesgo	Descartada	Expone a menores y convierte una alerta en estigma (Asamblea Legislativa de El Salvador, 2022, Art. 77)

4.4 Prototipar

Construimos dos prototipos. El primero es el prototipo funcional de datos: el pipeline, la base y el modelo que ya responden las preguntas de cada usuario. El segundo es un prototipo de baja fidelidad de las pantallas de la PWA (Figura 5), pensado para teléfonos de gama baja: una sola columna, texto grande y nada de imágenes pesadas.

Figura 5

Prototipo de baja fidelidad de las pantallas principales de la PWA



Nota. Elaboración propia. De izquierda a derecha: consulta con el número de estudiante, lista de tareas de la familia y panel de coordinación con alertas de riesgo.

4.5 Evaluar

Lo validado hasta ahora es técnico: los criterios de aceptación del Sprint 1 se comprueban con pruebas automáticas. La prueba de usabilidad con usuarios se hará en el Sprint 2, con el prototipo navegable de la PWA, según este protocolo:

Tabla 11

Protocolo de evaluación de usabilidad para el Sprint 2

Aspecto	Plan
Participantes	Cinco por perfil (familias, estudiantes y docentes), suficientes para encontrar la mayoría de los problemas de usabilidad (Nielsen, 2000)
Tareas	Encontrar las tareas pendientes con un número de estudiante; identificar la próxima fecha límite; en el panel, encontrar la materia con más retrasos
Equipos	Un teléfono Android de gama baja, un teléfono Huawei sin Google Play y una computadora con navegador antiguo
Métricas	Tasa de éxito por tarea, tiempo para encontrar lo pendiente (meta: menos de 1 minuto) y cuestionario SUS (meta: 68 puntos o más; Brooke, 1996)
Uso de resultados	Cada problema encontrado se convierte en un ítem del backlog con su prioridad

5. Evaluación tecnológica, económica y financiera

5.1 Viabilidad técnica

ATE es viable con tecnología gratuita durante el proyecto y con costos bajos en producción. Todo el pipeline corre en una laptop de 8 GB, y el mismo código pasa a la nube cambiando solo las variables de entorno. La API servirá el mejor modelo de scikit-learn, que pesa poco y no necesita TensorFlow, así que cabe en el plan más pequeño de cualquier proveedor.

Tabla 12

Infraestructura en local y en la nube

Componente	Desarrollo y exposición (USD 0)	Producción (desde el mes 1)	Costo mensual (USD)
Base de datos	MongoDB local o Atlas M0	MongoDB Atlas Flex; Atlas M10 desde el mes 7	30 → 57
API	Docker local; Render o Hugging Face Spaces gratis	Render Starter; Render Standard desde el mes 13	7 → 25
Frontend PWA	Vercel Hobby y GitHub Pages	Vercel Pro	20
Almacén de objetos	MinIO local o moto	AWS S3	2 → 5
Total de nube	USD 0	Según el tramo	59 → 107

Nota. Precios de lista consultados el 28 de octubre de 2026 en las páginas oficiales de cada proveedor (MongoDB, s.f.; Render, s.f.; Vercel, s.f.; Amazon Web Services, s.f.). El plan gratuito de Atlas (M0) no tiene respaldos ni acuerdo de servicio, por eso no se usa con clientes que pagan.

5.2 Supuestos del modelo de negocio

El modelo financiero está en `src/utils/finanzas.py`; todos los supuestos se pueden cambiar y volver a calcular con `python -m src.utils.finanzas`.

Tabla 13

Supuestos del flujo de caja

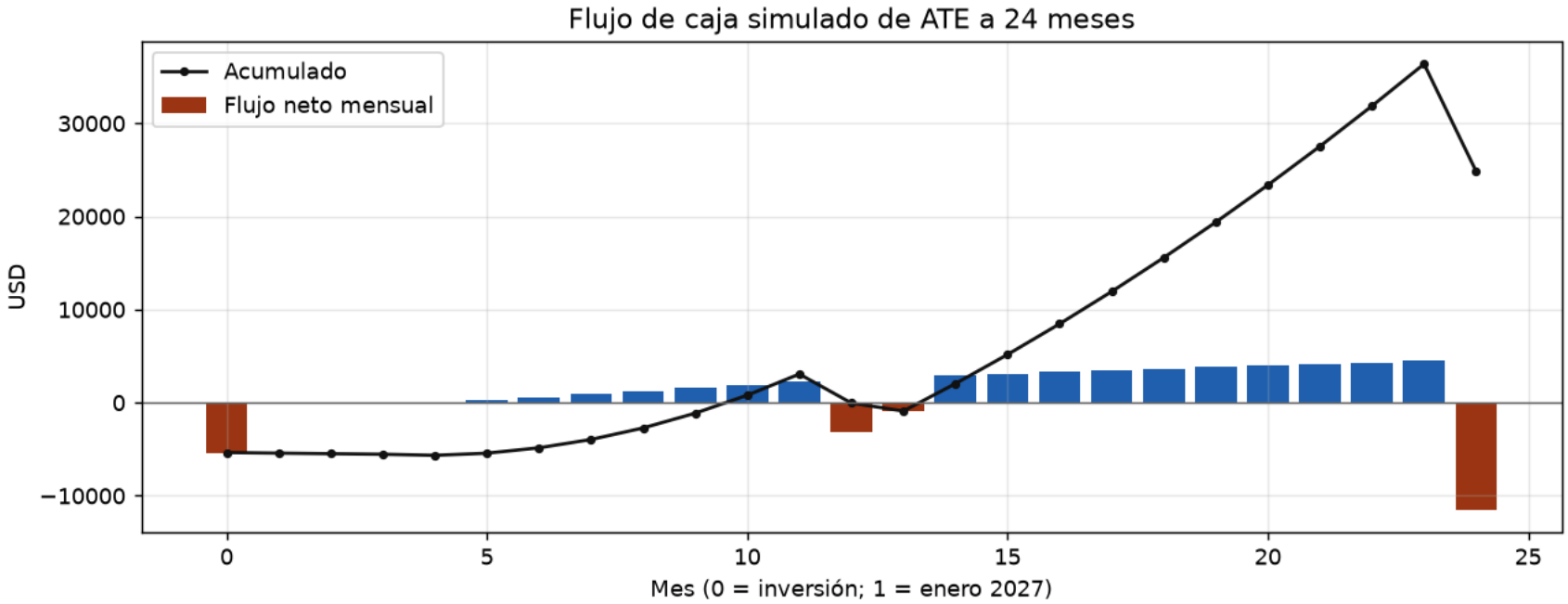
Supuesto	Valor
Precio	USD 0,50 por estudiante y mes lectivo (de febrero a noviembre)
Matrícula promedio por centro	400 estudiantes
Adopción	2 centros piloto gratis durante los 3 primeros meses; 20 centros al mes 12 y 32 al mes 24

Supuesto	Valor
Inversión inicial	USD 5 365: desarrollo (3 personas × 160 h × USD 10), revisión legal del aviso de privacidad, teléfonos de prueba y dominio
Costos operativos	Nube (Tabla 12), soporte de medio tiempo (USD 500) y contabilidad y mercadeo (USD 250) desde el mes 4
Comisiones	10 % de ventas y 3,5 % de pasarela de pago
Impuesto sobre la renta	30 % de la utilidad anual positiva (Asamblea Legislativa de El Salvador, 1991, Art. 41)
Tasa de descuento	12 % anual, para el valor actual neto (VAN)

5.3 Flujo de caja y ROI a 12 y 24 meses

Figura 6

Flujo de caja simulado de ATE a 24 meses (escenario base)



Nota. Las barras son el flujo neto de cada mes y la línea, el acumulado. Los meses sin ingresos corresponden al piloto gratuito y a las vacaciones de diciembre y enero.

Tabla 14

Indicadores financieros por escenario

Escenario	Horizonte	Ingresos (USD)	Egresos (USD)	Beneficio neto (USD)	ROI	VAN al 12 % (USD)	Mes de recuperación
Pesimista (50 % de la adopción)	12 meses	8 800	14 179	−5 379	−100 %	−5 416	No se recupera
Pesimista (50 % de la adopción)	24 meses	35 400	31 872	3 529	66 %	2 171	19
Base	12 meses	17 600	17 646	−46	−1 %	−422	No se recupera
Base	24 meses	70 600	45 754	24 846	463 %	20 830	14
Optimista (125 % de la adopción)	12 meses	22 000	19 382	2 618	49 %	2 076	9
Optimista (125 % de la adopción)	24 meses	88 400	52 776	35 624	664 %	30 264	9

Nota. ROI = beneficio neto del período ÷ inversión inicial; el beneficio neto ya descuenta la inversión. Fuente: reports/escenarios_financieros.csv.

En el escenario base, el primer año queda prácticamente en equilibrio: el piloto gratuito y las vacaciones de diciembre y enero dejan cinco meses sin ingresos. El segundo año, con 20 a 32 centros, la inversión se recupera en el mes 14 y el ROI llega a 4,6 veces. Incluso con la mitad de la adopción esperada, el proyecto recupera la inversión en el mes 19. El riesgo financiero principal no es la tecnología, que cuesta menos de USD 110 al mes, sino la velocidad con que los centros adoptan la plataforma.

6. Riesgos y próximos pasos

6.1 Riesgos actualizados

Tabla 15

Riesgos al cierre del Sprint 1

Riesgo	Cambio desde la propuesta	Mitigación
Poco tiempo para el Sprint 2	Aumentó: el Sprint 1 se extendió al 2 de noviembre	Priorizar la API, Docker y el README sobre el frontend
Memoria de las laptops del equipo	Nuevo	Ejecutar Spark con el *driver* de 1 GB y los datos fuera de OneDrive; la imagen Docker no incluye TensorFlow
Historial de Git con un solo autor visible	Nuevo	Publicar el repositorio en GitHub e invitar a los tres integrantes; repartir las historias del Sprint 2 con *pull requests* revisados
Acceso a Atlas desde equipos nuevos	Nuevo	Agregar la IP de cada equipo en *Network Access*; la demo tiene respaldo local
Datos sintéticos no representan una escuela real	Igual	Resultados presentados como orientativos y limitaciones escritas en el README

6.2 Plan del Sprint 2

Tabla 16

Historias del Sprint 2 y responsables

Historia	Entregable	Responsable principal
HU-12	API FastAPI con `/health` y `/v1/predict`, Dockerfile y `.dockerignore`	Mauricio Castro
HU-01, HU-02, HU-04	PWA en React: consulta de tareas, registro de entregas y alertas de riesgo	Mauricio Castro
HU-14	README con los diez apartados de la guía, tabla de trazabilidad y declaración de uso de IA	Claudia Carballo
HU-11 (cierre)	Revisión de *pull requests*, pruebas de la API y cobertura	Claudia Carballo
HU-17, HU-18	Calendario en la API, aviso de privacidad y procedimiento ARCO-POL	Julio Flores
HU-13	Publicación en Vercel y Render, y carga de la muestra en Atlas	Julio Flores
v1.0.0	Etiqueta de la versión final el 6 de noviembre	Equipo

7. Declaración de uso de inteligencia artificial

Como permite la guía del proyecto (Bootcamp de Python para Ciencia de Datos, 2026b, sección 12), el equipo usó Claude (Anthropic, 2026) como asistente en estas tareas:

- **Programación:** escribir y revisar módulos del pipeline, las pruebas pytest y los scripts de entrenamiento.
- **Depuración:** diagnosticar los problemas de memoria y de conversión de PySpark descritos en la Tabla 3.
- **Documentación:** redactar y dar formato APA a este documento, al README y al registro de decisiones.

Las decisiones del proyecto, los supuestos financieros y la revisión de los resultados son del equipo. Cada integrante puede explicar y modificar cualquier parte del código y del documento. Todas las cifras de este documento salen de ejecuciones reales registradas en `reports/`.

Referencias

Amazon Web Services. (s.f.). *Amazon S3 pricing*. Recuperado el 28 de octubre de 2026, de https://aws.amazon.com/s3/pricing/

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje grande]. https://claude.ai

Asamblea Legislativa de El Salvador. (1991). *Ley de Impuesto sobre la Renta* (Decreto Legislativo N.° 134). https://www.asamblea.gob.sv/decretos/details/310

Asamblea Legislativa de El Salvador. (2022). *Ley Crecer Juntos para la Protección Integral de la Primera Infancia, Niñez y Adolescencia* (Decreto Legislativo N.° 431). https://www.unicef.org/elsalvador/media/4611/file/Ley%20Crecer%20Juntos.pdf

Asamblea Legislativa de El Salvador. (2024, 15 de noviembre). *Ley para la Protección de Datos Personales* (Decreto Legislativo N.° 144). Diario Oficial. https://www.jurisprudencia.gob.sv/DocumentosBoveda/D/2/2020-2029/2024/11/10660E.PDF

Bootcamp de Python para Ciencia de Datos. (2026a). *Guía de planificación, entregables y cronograma* [Material del curso].

Bootcamp de Python para Ciencia de Datos. (2026b). *Guía del Proyecto Integrador: Del dato crudo al servicio desplegado* [Material del curso]. Módulo 4: Big Data y MLOps.

Brooke, J. (1996). SUS: A "quick and dirty" usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester y I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

Hasso Plattner Institute of Design at Stanford. (2010). *An introduction to design thinking: Process guide*. https://web.stanford.edu/~mshanks/MichaelShanks/files/509554.pdf

MongoDB. (s.f.). *MongoDB Atlas pricing*. Recuperado el 28 de octubre de 2026, de https://www.mongodb.com/pricing

Nielsen, J. (2000, 18 de marzo). *Why you only need to test with 5 users*. Nielsen Norman Group. https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Render. (s.f.). *Pricing*. Recuperado el 28 de octubre de 2026, de https://render.com/pricing

Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling. En K. Huff y J. Bergstra (Eds.), *Proceedings of the 14th Python in Science Conference* (pp. 126–132). https://doi.org/10.25080/Majora-7b98e3ed-013

Vercel. (s.f.). *Pricing*. Recuperado el 28 de octubre de 2026, de https://vercel.com/pricing

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S. y Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM, 59*(11), 56–65. https://doi.org/10.1145/2934664